

Projeto de automação e controle

2D Plotter

Controle cartesiano com Arduino Uno e Wokwi

Responsável: [Sergio Cordero Calvimontes](#)
Plataforma: Arduino Uno R3
Linguagem: Arduino C++
Simulação: Wokwi

12 de agosto de 2026

Sumário

1	Introdução	2
2	Objetivos e requisitos	2
3	Arquitetura do sistema	2
4	Controle de movimento	3
4.1	Joystick e perfis X/Y	3
4.2	HOME e limites	3
5	Caneta, telemetria e sinalização	3
6	Saídas auxiliares	3
7	E-STOP e análise de segurança	4
8	Verificação	4
9	Conclusão	4
	Referências eletrônicas	4

1 Introdução

Este projeto desenvolve o controle simulado de uma plotter cartesiana de dois eixos. O sistema reúne acionamento de motores de passo, posicionamento da caneta, interação por joystick, telemetria, sinalização de velocidade, cargas auxiliares e funções de parada. A simulação executável está disponível em <https://wokwi.com/projects/472168224663835649>; os arquivos `sketch.ino`, `diagram.json` e `libraries.txt` tornam a implementação reproduzível.

Além do funcionamento, a organização do repositório separa fontes, documentação bilíngue, verificações e artefatos de relatório. Essa separação facilita auditoria entre requisitos, pinagem, diagrama e firmware.

2 Objetivos e requisitos

O objetivo principal é comandar os eixos X/Y com movimento acelerado e limites lógicos, enquanto a caneta assume três alturas. Como requisitos complementares, o sistema deve apresentar a posição em OLED, indicar a velocidade dos eixos, controlar sucção e ventilação e reagir imediatamente ao E-STOP.

Tabela 1: Principais parâmetros do projeto.

Parâmetro	Valor
Limites X/Y	0–4000 / 0–2400 passos
Velocidade e aceleração	800 passos/s; 450 passos/s ²
HOME	retorno lógico a (0,0), até 500 passos/s
Z	alta 180°, meia 90°, baixa 0°
OLED	SSD1306 128×64, I ² C, endereço 0x3C
Barras	duas barras de dez segmentos, três 74HC595
Retenção do ventilador	2000 ms após movimento

3 Arquitetura do sistema

O Arduino Uno centraliza entradas, geração de passos e atualização das saídas. A Figura 1 apresenta a divisão funcional.

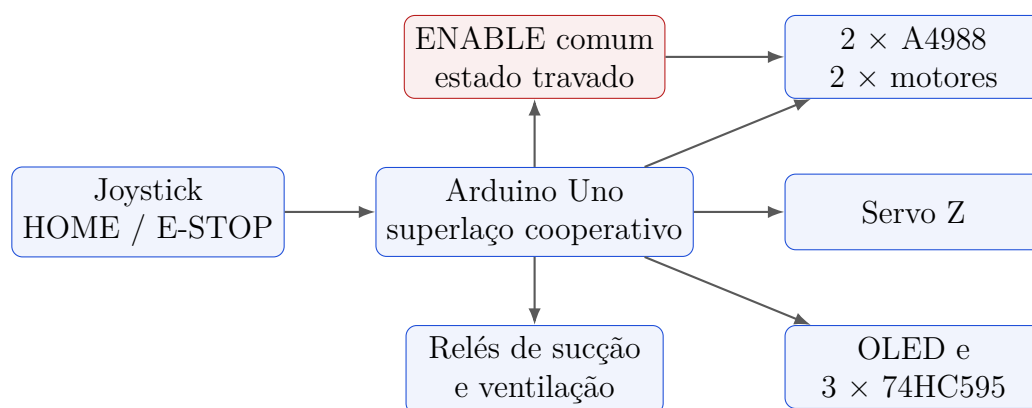


Figura 1: Arquitetura funcional do controle e dos periféricos.

O firmware evita `delay()` porque a biblioteca `AccelStepper` exige chamadas frequentes a `run()`. Entradas e estados são processados em ordem de prioridade; barras e OLED usam temporizadores próprios de 30 e 80 ms.

4 Controle de movimento

4.1 Joystick e perfis X/Y

Cada eixo analógico possui centro nominal 512 e zona morta de 70 unidades. A distância além da zona morta é normalizada até a velocidade máxima. Um comando positivo define o limite máximo como alvo; um comando negativo define o mínimo. Ao centralizar, `stop()` calcula a desaceleração e o alvo é novamente limitado à área útil.

Listing 1: Conversão resumida da entrada analógica.

```
1 float joystickToSpeed(int raw) {  
2     if (raw > JOY_CENTER + JOY_DEADZONE) { /* escala positiva */ }  
3     if (raw < JOY_CENTER - JOY_DEADZONE) { /* escala negativa */ }  
4     return 0.0f;  
5 }
```

4.2 HOME e limites

HOME eleva a caneta, encerra a sessão de trabalho e move os dois eixos para zero. Esse processo é não bloqueante e somente termina quando distância e velocidade de ambos os eixos chegam a zero. Como não existem chaves físicas, a operação depende da posição interna e não corrige perda de passos.

5 Caneta, telemetria e sinalização

O seletor do joystick alterna a caneta entre alta, meia e baixa, com antirrepique de 35 ms. O OLED representa a área útil e converte os passos em coordenadas gráficas. O marcador muda de forma conforme a altura da caneta e as bordas são realçadas quando os limites lógicos são atingidos.

A velocidade real de cada eixo, após a rampa de aceleração, é convertida em rpm e em nível de zero a dez. Três 74HC595 em cascata distribuem os vinte bits das barras usando apenas DATA, CLOCK e LATCH.

6 Saídas auxiliares

Movimento manual ou alteração de Z inicia uma sessão de trabalho. A sucção permanece ativa nessa sessão e é desligada quando HOME começa. O ventilador acompanha qualquer movimento ou HOME e permanece ligado por dois segundos após a parada. Na simulação, LEDs após os contatos dos relés representam as cargas.

7 E-STOP e análise de segurança

E-STOP é lido antes de qualquer outra função e sem antirrepique. Quando ativo, o programa coloca o ENABLE comum em nível alto, desliga as cargas auxiliares, zera as barras, mostra uma mensagem de emergência e não retorna à operação sem reset.

Essa reação é adequada para demonstrar a lógica, mas não constitui função de segurança certificada. Uma plotter física exige E-STOP cabeado, remoção segura de energia, fins de curso, proteção contra partes móveis, fonte apropriada de motores, limitação de corrente e análise de riscos.

8 Verificação

A suíte Python verifica a estrutura do diagrama, contagens, identificadores, ligações críticas, constantes de pinagem, bibliotecas e propriedades do laço principal. A execução é:

```
python -m unittest discover -s tests -v
```

Os testes estáticos devem ser complementados no Wokwi com movimentos nos quatro sentidos, três estados de Z, HOME, temporização das cargas, limites e E-STOP durante movimento. A compilação para Arduino Uno continua sendo um critério separado.

9 Conclusão

O projeto integra 31 elementos e 113 conexões em uma arquitetura coerente com as restrições do Arduino Uno. O uso de controle não bloqueante mantém geração de passos, interface e auxiliares responsivos. Limites lógicos e E-STOP travado melhoram o comportamento da simulação, enquanto a documentação torna explícito que uma implementação física requer referenciamento e proteções adicionais.

Referências eletrônicas

- Projeto no Wokwi: <https://wokwi.com/projects/472168224663835649>
- Arduino Uno Rev3: <https://docs.arduino.cc/hardware/uno-rev3/>
- AccelStepper: <https://www.airspayce.com/mikem/arduino/AccelStepper/>
- Adafruit SSD1306: https://github.com/adafruit/Adafruit_SSD1306
- Formato de diagrama Wokwi: <https://docs.wokwi.com/diagram-format>